```
  throw-if (elem == null or not
parentsMap.has(elem)))                          #2
  parent =
this.parentsMap[elem]
     #3
  if parent != elem
then                                                    #4
parent =
this.findPartition(parent)
       #5
  return
parent
      #6
```

#1 Takes an element and return another element, the one element at the root of the tree for the partition to which `elem` belongs.
#2 Check that the element is valid.
#3 Retrieve the parent of the element.
#4 If current element's `parent` is `elem` itself, then we already got to the root of the tree, otherwise…
#5 … we need to climb up recursively to the root, by looking for `parent`'s partition.
#6 At this point `parent` stores the root of the tree for the partition containing `elem`, so we can return it.

After getting the element's parent, we check to see if it's the element itself: if an element is its own parent, then we know this means we reached the root of the partition's tree, because of the way we initialize this field, and because, in `merge`, we never change a root's parent.

Otherwise, if current element does have a parent, we walk up the tree, towards its root, and perform a recursive call to